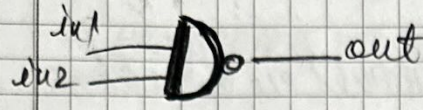
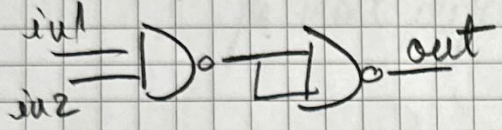
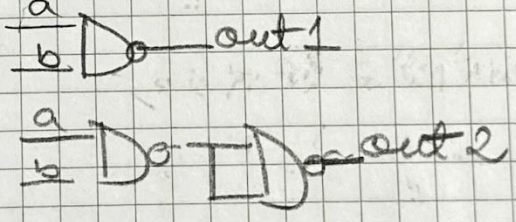
SOC CURS 2 4.03.2024




```
module NAND (in1, in2, out)
  input in1, in2;
  output out;
  assign out = ~(in1 & in2);
endmodule
```




```
module AND (in1, in2, out)
  input in1, in2;
  output out;
  wire w;
  NAND n1 (in1, in2, w);
  NAND n2 (w, w, out);
endmodule
```




```
module testAND
  reg a, b;
  wire out1, out2;
  initial begin // → date pt test
    a = 0; b = 0;
    #1 a = 1 → delay ! #1 b=1 => a=1 b=1
  end
  initial begin
    $monitor
```

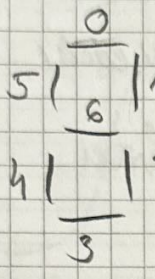
unit time

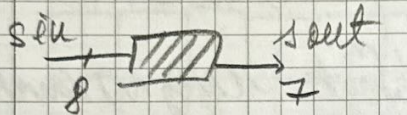
optional

III } AND gate1(a,b,out2)
NAND gate2(a,b,out1) / instantiere module
endmodule

In always numai reg

binar → hexa (Afisare 7 segment)





input [7:0] sin;
output [6:0] sout;
reg [6:0] sout;
→ am copiat
output reg [6:0] sout;

sout

6	5	4	3	2	1	0
0	0	0	0	1	1	0

↳ sunsin pt 1 la afisaj

always @(sin);

6	5	4	3	2	1	0
1	0	1	1	0	1	1

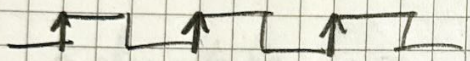
case(sin)
pt1: 8'b00000001 : sout =
= 7'b0000110;
pt2; 8'b00000010 : sout =
= 7'b1011011;
:
default: sout = 7'b0111111;
endcase
endmodule

SOC CURS 3 11/03/24

- expresie eveniment @
- wait
- comanda intarziere

initial begin
a = 0 ; b = 1;
#20 a=1; b=0
↳ după 20 unit de timp
end

always @(posedge clk) → numai pe front pozitiv de ceas



forever vs always → în modul
↳ potrivit pentru bucla de inițial

CLR - clear (1, 0)
CLK - clock (↑, ↓, x)

1 - CLR
x - CLK
=> semnal asincron

module ports
reg → output
SAU output reg [3:0] q; → pt că folosim always
[3:0] q

BUCLA SEPARATĂ PT ASINCRON
always @(clr)
q <= 0
aici nu depinde de ceas

end
endmodule

dacă ~~este~~ NU AM × pt CLK => 1 always

```
always @(posedge clk)
begin
   if(dr)
      q<=0;
   else if(ce)
      q<=d;
end
```

```
always@(a,b,c)
begin
   d=a^b;
   e=b^c;
end
```

I

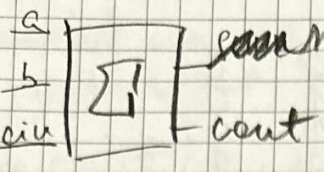
SAU

I

assign d=a^b
assign e=b^c

la assign nu mai e nevoie de reg

~~out~~ reg
~~in~~ wire) pt testare



sum = a^b^cin

cout = ab

(cout, sum) = a+b+cin

↳ la nivel de un bit în modul

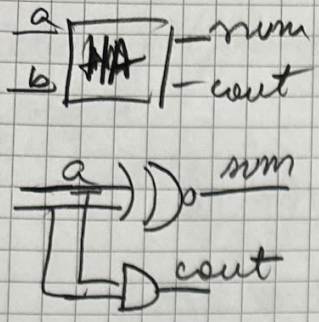
```
module
  input
  output
  reg → necesită reg
  always@(a or b or cin)

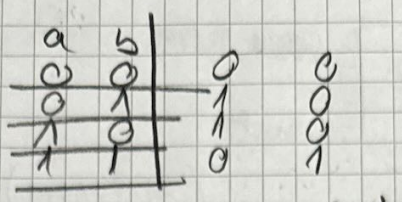
endmodule

nmator inst (a, b, cin, sum, cout) //instanțiere

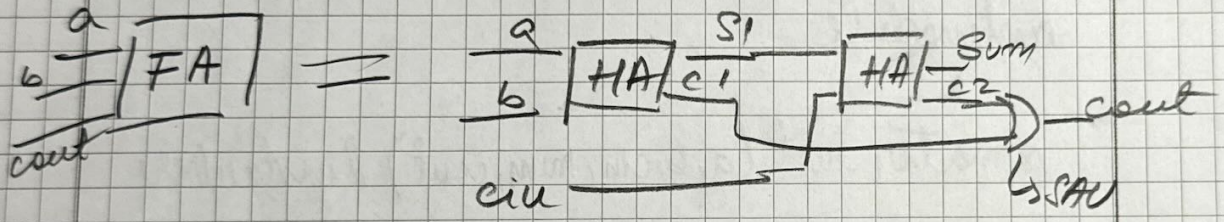
initial begin
  .
  #10 a=1; b=1; cin=1
  #10 $finish
end

initial
  $monitor $time ("suma %b transport %b", sum, cout)
                 ↳ consola
```






```
module ha(a,b, sum, cout)
  input a;b;
  output sum, cout;
  assign sum = a ^ b;
  assign cout = a & b;
endmodule
```




```
module fa(a,b,cin, sum, cout)
  input a,b,cin;
  wire c1,s1
  output sum, cout;
  ha m1 (a,b,s1,c1)
  ha m2(s1, cin, sum, c2)
  assign cout = c2|c1;
```

FARĂ ALWAYS LA STRUCTURAL

TEST

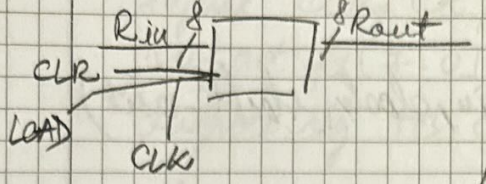
I reg a,b,cin
wire cout, sum

II FA Mg(a,b,cin,sum,cout)

III initial begin
a=0; b=0; cin=0;
#20

```
module max (f, a, b, sel)
  input a, b, sel;
  output reg, f;
  always @(a, b, sel)
  case(sel)
    0 : f = a;
    1 : f = b;
  end ~~module~~ case
endmodule
```

SOC CURS 18/03/24



parameter width = 8
input [width-1:0] a, b;

Keyword

input clear, load, clock
(CLR) (CLK)

! SECVENȚIAL => always @

always @ (posedge clock)

! IEȘIRI → OUTPUT
↳ REG

INTRĂRI → INPUT
↳ WIRE (?)

```
if (clear)
    rout <= 0;
else if (load)
    rout <= rin;
```

↳ neblocant: atribuirea se face la final

→ secvențial => am nevoie de DELAY

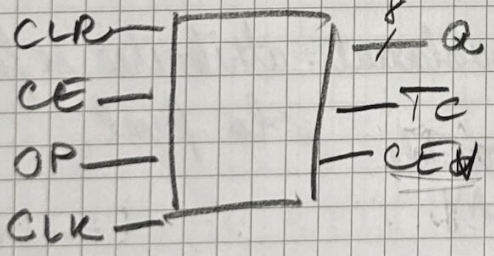
initial begin
DAU VALORI
ȚIN CONT DE CEAS
clock = 0; clear = 0; load = 0; rin = 19; → intrarea
MODIFIC VALORILE ȘI TESTEZ COMBINAȚII
#20 load = 1; → trece peste astea sau $ finish
#10 rin = 14;
#30 clear = 1; $ stop
end

perioada ceasului

! DOAR PT SECVENȚIAL
always
#20 clock = !clock

SHIFT REGISTER

```
module shiftReg(shift_in, clock, shift_out)
  input shift_in, clock;
  output shift_out
  reg bit0, bit1, bit2, bit3
  assign shift_out = bit3;
  always @(posedge clock) begin
    bit3 <= bit2
    bit2 <= bit1
    bit1 <= bit0
    bit0 <= shift_in
  end
endmodule
```



input				output		
CLR	CE	OP	CLK	Q[7:0]	TC	CEO
1	X	X	↑	0	0	0
0	0	X	↑	NO CHANGE	NO CHANGE	0
0	1	0	↑		TC	CEO
0	1	1	↑		TC	CEO

```
module circuit(CLR, CE, OP, CLK, Q, TC, CEO)
 input CLR, CE, OP, CLK;
 output reg Q[7:0];
 output reg TC, CEO;
 always @(posedge CLK) begin
   if (CLR)
     begin Q=0; TC=0; CEO=0; end
   else
     begin
       if (CE and ~OP)
         begin Q=Q-1; TC=1; CEO=1; end
       else begin (CE and OP)
         end Q=Q+1; TC=1; CEO=0; end
```

MEALY / MOORE

MOORE

2 biți => 3 stări

reg [1:0] state, nextstate

parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10;

always @(posedge clk, posedge reset) → SECVENȚIAL

always @(*) → COMBINAȚIONAL

case (state)

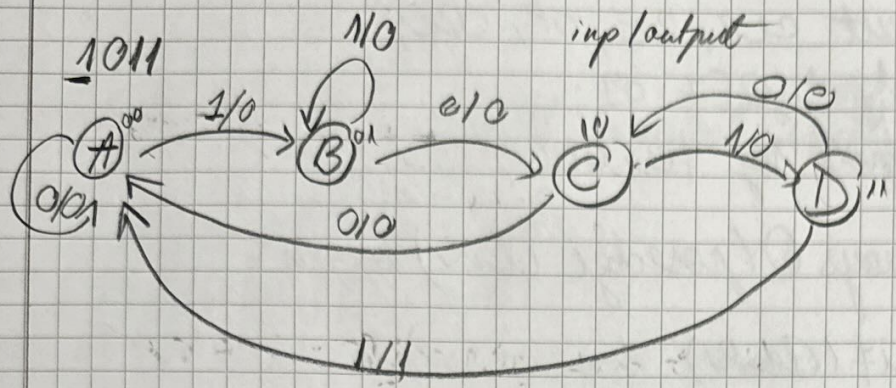
S0: nextstate = S1

default: next state = S0

assign out = (state == S0)

MEALY

1011




```
module bd (in, clk, reset, out)
  input in, clk, reset
  output reg out;
  reg [1:0] state next+state
  always @ (posedge clk) begin
    if (reset) state <= 2'b00;
    else       state <= next+state
  end
  always @ (state, in)
    case ( {state, in} )
                                          (A->A)
      3'b 00 0 : begin next+state : 2'b00;
                 out = 0;
                 end
                                          (A->B)
      3'b 00 1 : begin next+state : 2'b01
                 out = 0;
                 end
                                          (B->C)
      3'b 01 0 : begin next+state : 2'b 10
                 out : 0;
                 end
                                          (B->B)
      3'b 01 1 : begin next+state : 2'b 01
                 out = 0;
                 end
```

(C → A)

3'b100: begin nextstate = 2'b00;
out = 0;
end

(C → D)

3'b101: begin nextstate = 2'b11;
out = 0;
end

(D → C)

3'b110: begin nextstate = 2'b10;
out = 0;
end

3'b111: begin nextstate = 2'b00;
out = 1;
end

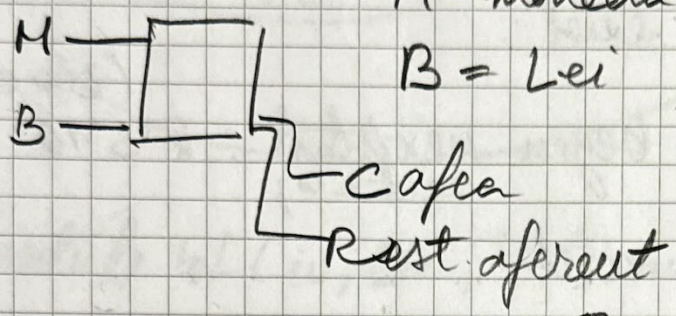
default: nextstate = 2'b00

endcase
endmodule

SOC CURS 21/03/24

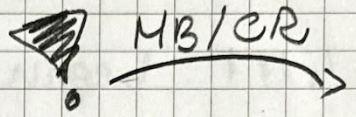
AUTOMAT DE STARI

automat cafea (Mealy)

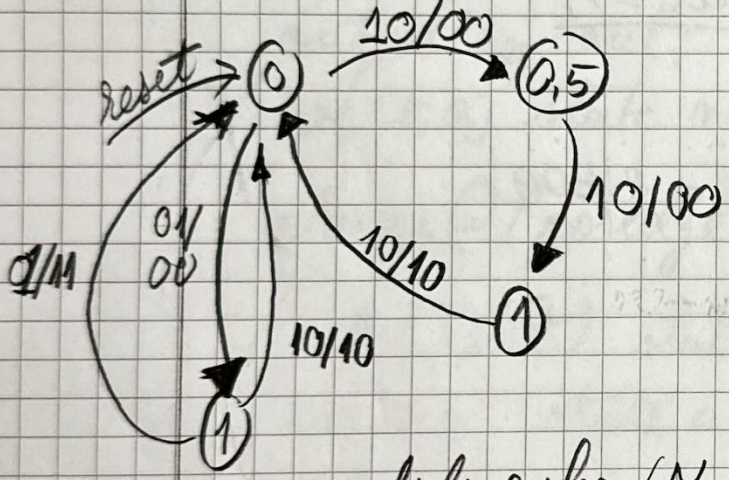


M = moneda = 0,5

B = Lei ; Cafea = 1,5 Lei



Stari




```
module cafea(M,B,clk,reset,C,R)

input M,B,clk,reset;
output reg C,R;
reg [1:0] state, next_state;

always @(posedge clk)
  if(reset)
     state <= 2'b00;
  else
     state <= nextState;
```

```
always @(state, M, B)
    case ({state, M, B})

        4'b 0010 : begin
            nextstate = 2'b01;
            C = 0; R = 0;
        end

        4'b 0000: begin
            nextstate = 2'b 00; C = 0; R = 0;
        end

        4'b 0110: begin
            nextstate = 2'b 10; C = 0; R = 0;
        end
        si tot așa mai departe
```

TESTARE

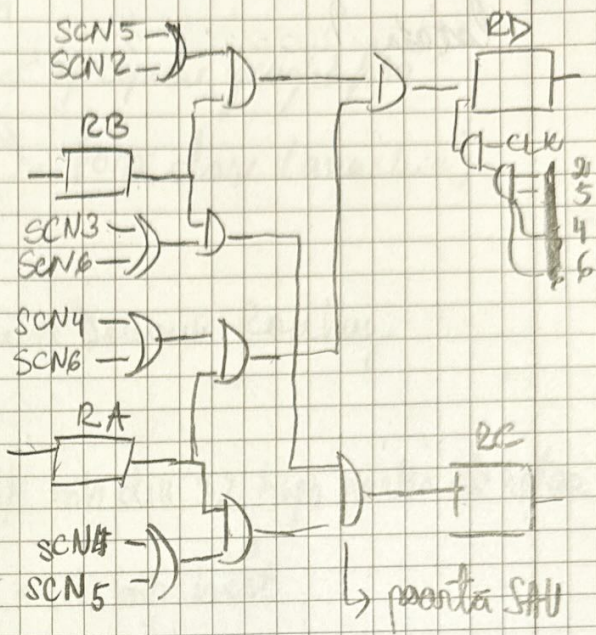
```
initial begin
    clk = 0; reset = 1; M = 0; B = 0;
    #20 reset = 0;
    #20 M = 0; B = 1
    #20 M = 0; B = 1
    #20 M = 0; B = 1
    #20 M = 1; B = 0

always #10 clk = !clk
```

Curs SOC 25/03/24

RA, RB / RC, RD

1. RC = RA
2. RD = RB
3. RC = RB
4. RD = RA
5. RC = RA, RD = RB
6. RC = RB, RD = RA



SCN = Semnal de Comanda de Nivelare

Porțile ȘI = nr reg. surse · nr reg dest · nr biți ai reg

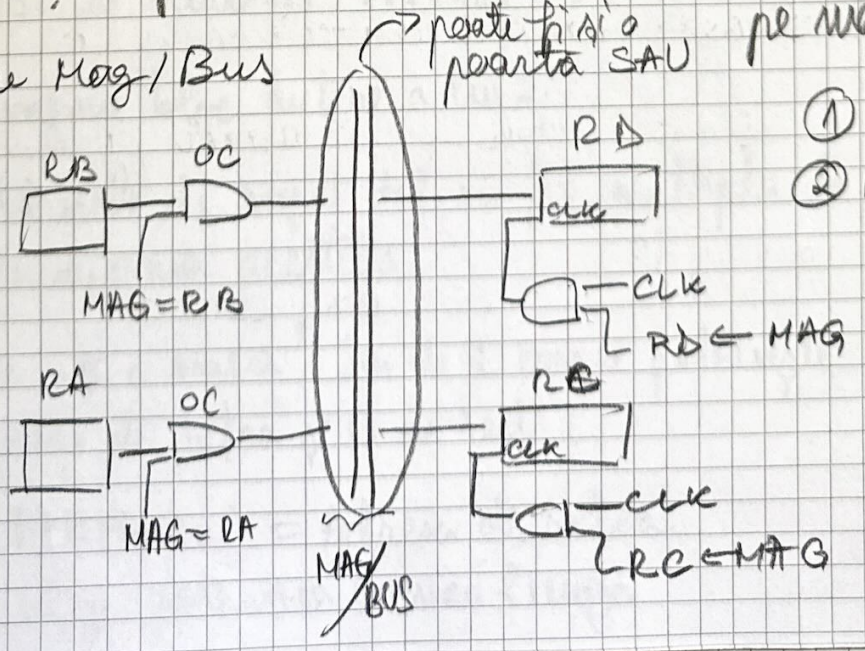
= 2 · 2 · 1 = 4

Porți SAU = nr reg dest · nr biți = 2 · 1 = 2

-> Per total 6 porți

Soluția optimă (și secvențială) este transferul pe magistrală

Cu Mag / Bus



① MAG ← RA

② RC ← MAG

Cost

ȘI = 4 · 2
= 2 · 1

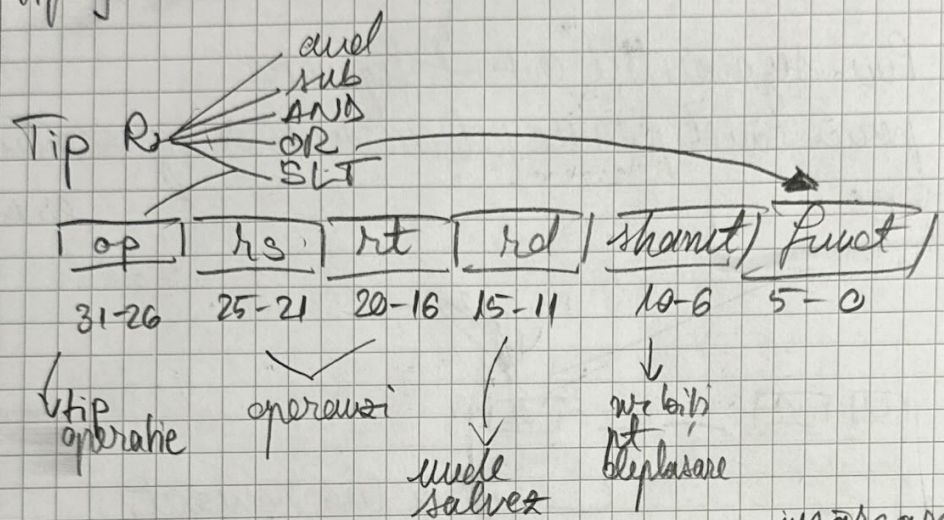
SAU = 1
= 1

3 porți

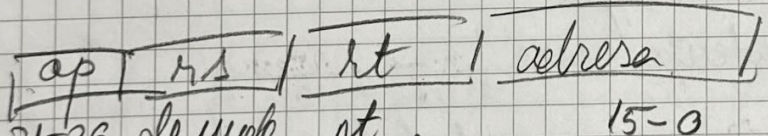
SOC 8/04/24

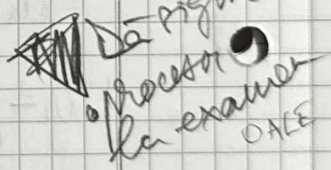
Mod de Adresare

Tip R - aritmetice
Tip I - transfer, ramificație, imediat
Tip J - salt



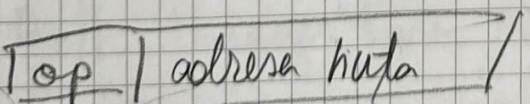
Tip I (lucru cu memoria) → încărcare în memorare în ramificare beq

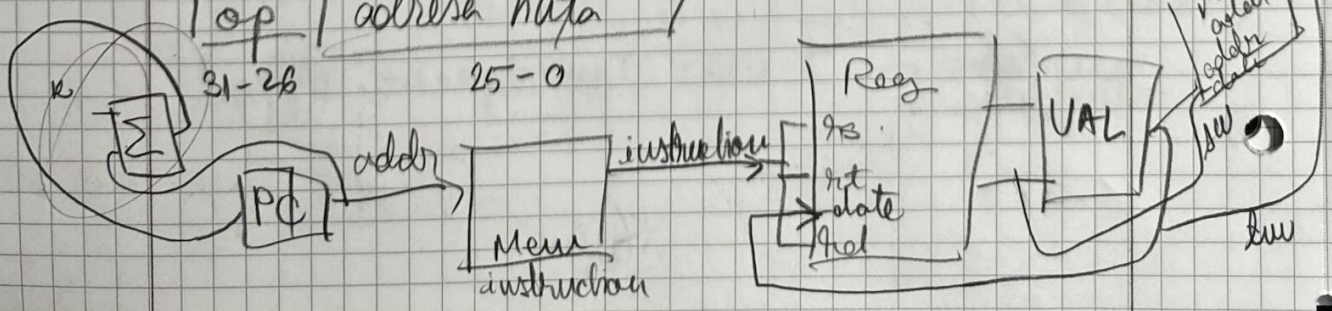




PC - program counter

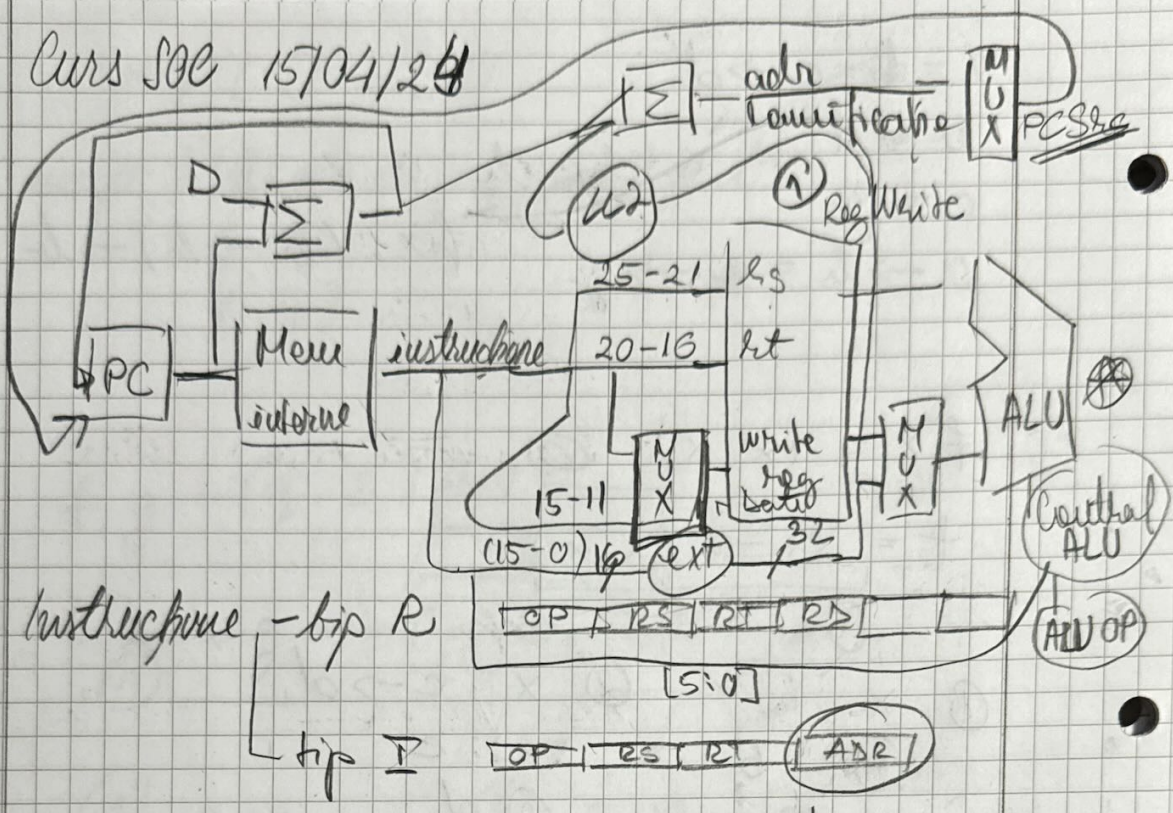
Tip J → jump



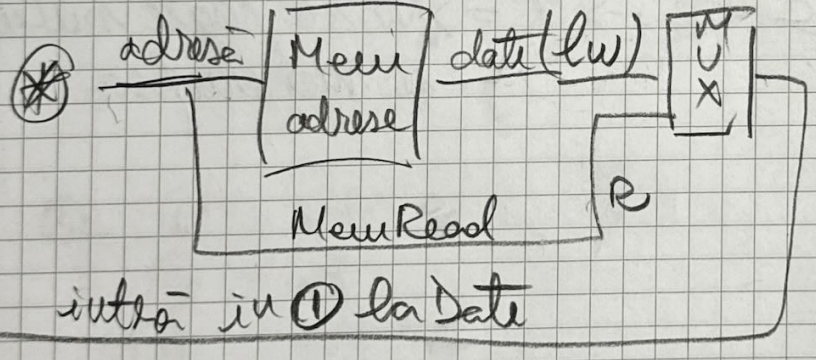


Curs SOC 15/04/26

OALE



Mem Unit



ieșire în (1) la Date

Is ieșire în unitate extindere semn

ALU Op
- 00 lw/sw
- 01 Beq
- 10 R

[Tip]

Curs SOC 18/04/24

Semnale de control - Mem Read
- Mem Write
- Reg Write
- ALU Control
- Instruction Register Write

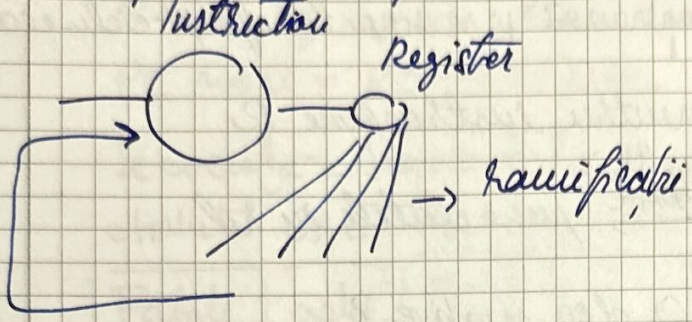
PC - record → jump
- conditional → Write Cond (semnal control)

Unitate Control

IOD - ? (2e-16) → both pt scalcare

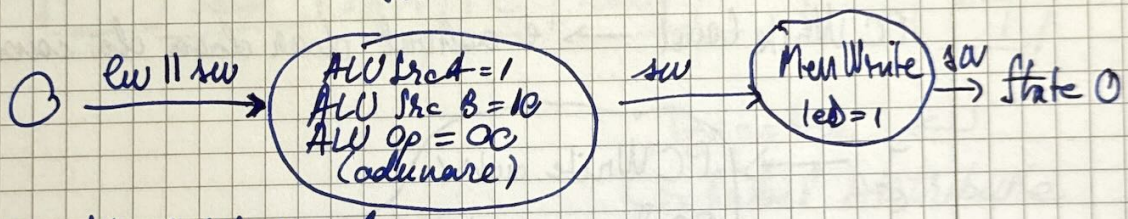
Curs SOC 22/04/24

Mealy vs Moore după al controlului



Adresa va fi memorată pt eficiență

O ramificație [lw / sau / sw



I | rs | rt | adr |

MemWrite ia date din B și le pune ce (ieș = 1) ?

RegWrite = 1 → scrie în regiștrele generale

MemToReg acționează ca multiplexorul care mută adresele din date în regiștre (rt)

„Ce semnale activăm pentru instrucțiune de tip I?"

Op = R-Type (Aritmetică și logică)

ALU Src A = 1

ALU Src B = 00

A și B conțin operanzii și se scriu la sfârșit ciclu ceas

ALUOp = 10 pentru instrucțiune R

ALU Out ~~Result~~ RegWrite pune datele în Rd

MemToReg e 0 dar motive?

RegDest = 1 ca să ia biții destinație

Jump → Necondiționat

PC Write Cond → e activat doar dacă dă cond 0

J → PC Write activ
PC Source = 10
26 biți >> 2 și se lipește de PC+4

Mașină Moore?

7 câmpuri - etichetă / micro instr MIPS nou

- UAL
- SRC 1 și 2
- Control Reg
- Control Mem
- Control PC
- câmp de secvență / una instrucțiune micro/MIPS

Fetch → MIPS mare Sorl Src 2

PC+4 → UAL → adunarea [PC|+|4]

N-am Reg, nu am Mem

PC nou = PC+4 ∈ UAL

Secvență = Secvențial (urm microinstrucțiune)

etichete	UAL	SRC1	SRC2	Reg	Mem	PC	Secvență
Fetch	add	PC	4			UAL	Seq

rt + adresa (16 → 32) << 2 adresă pt racordificații

	add	Shift	PC	read			Seq

Dispatch ←

↳ Tabel Distribuție

MemI Rformat

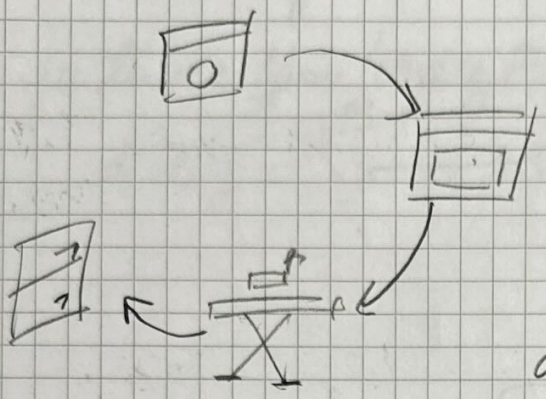
Excepții

Se salvează într-un registru care determină cauza

Din cauze ↑ → se setează bitul 1 pt instr nedef/overflow

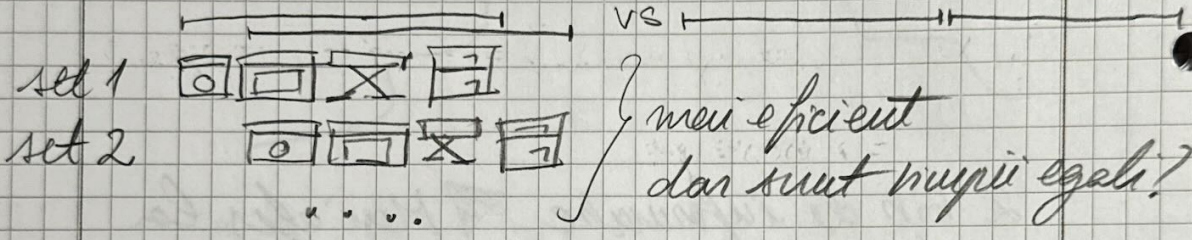
SOC 29/04/24

Ciclu spălare lucruri



Timp total 2 ore

Vreau un pipeline pt a eficientiza procesul



} mai eficient dar sunt nu pași egali?

Le iau cel mai lung proces și ala devine unitatea de timp. Pipeline => Throughput ↑

Stagii pipeline MIPS

1. Citire instr din mem de instr
2. Citire regiștrelor în timpul decodificării instr
3. Execuția op / calcul adresă (UAL)
4. Accesarea operand în mem de date
5. Scriere rezultat în registru

	Citire op	Citire reg	Op ALU
lw	200ps	100ps	200ps

Acces date	Scriere reg	⟶ Total
200ps	100ps	800ps

sw nu mai scrie în reg ⇒ 700 ps

Op R Citire rg(rs, rt) + Op ALU + Scriere reg

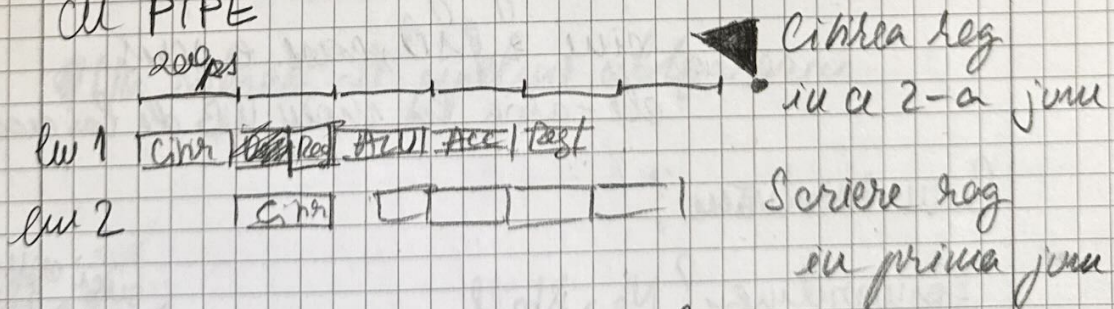
⇒ 600 ps

beq Citire op + Citire rg + Op ALU ⇒ 500 ps

TIMPI FĂRĂ TIMELINE → Maxim 800ps ⇒

→ Unit time pipeline = 800 ps

CU PIPE



FĂRĂ PIPE: 2400 ps 3× lw

CU PIPE: 1300 ps 3× lw

Hazard structural IMPOSIBIL la MIPS

Hazarduri

Il De date

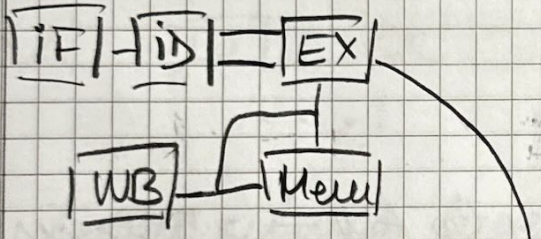
"Urm instr. nu se poate executa"

add $s0, $t0, $t1
op1 op2
→ op1 + op2

IF = instr fetch

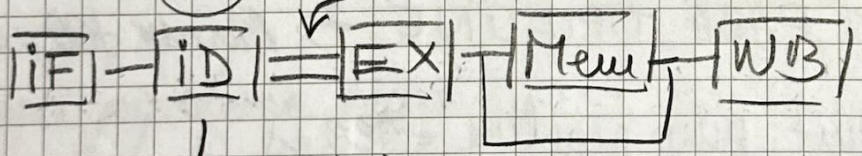
S+1 = IF
S+2 = ID (decode)
S+3 = execute = op ALU
S+4 = Mem = lucru cu date
S+5 = Write Back = înapoiere reg.

Stagii



Forwarding (Că am acces)

sub $t2, $s0, $t3



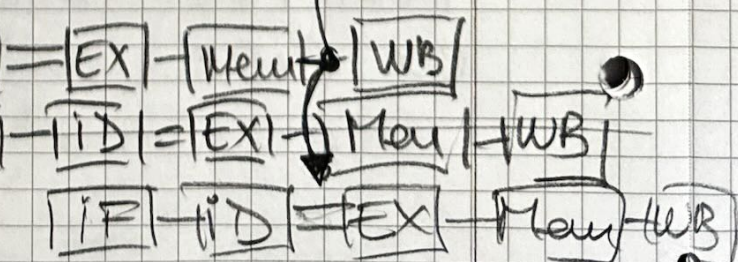
→ aici

când ajung încă nu știu cine e $s0, care e scris de-abia la mem WB de la add

Cum rezolvăm?

Forwarding? Nu, Stall

lw $s0, 20($t1)
sub $t2, $s0, $t2



Exemplu:

A = B + E B → $t0 → t1

C = B + F E $t0 → t2

E lw $t1, 0, $t0

B lw $t2, 4, $t0

\+ add $t3, $t1, $t2

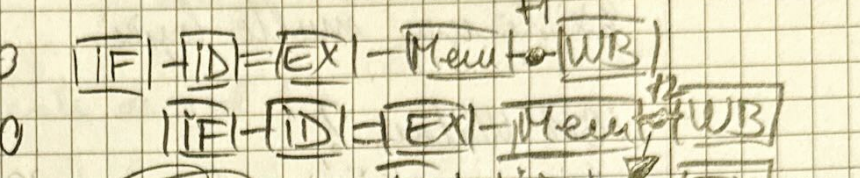
→ sw $t3, 12($t0)

F lw $t4, 8($t0)

\+ add $t5, $t1, $t4

→ sw $t5, 16($t0)

swap care ar intalzia natural



III Hazard de control

beq poate cauza asta din cauza branch prediction

1. Predicția nu e luată în calcul
2. Predicția e luată în considerare

=> timp de așteptare (≈ un ciclu de ceas)

! Un subset de pipeline obligatoriu

SOC 13/05/24

Pipeline ♥

Instrucțiuni → aritmetice - logice
→ load / store
→ ramificație

Registre de pipeline care țin minte datele altor stagii => Paralelizare

16 → 32 biți ∈ ID

ALU / calcul adresă ∈ EX

MEM: lucru cu memo cache a CPU-ului
(Data Memory în desen)

Fiecare stagiu are resurse HW personale

Flux •——→•, dar sunt și excepții:

1. Write Back, mă întorc să scriu în reg gen
2. Ramificație condiționată PC+4 / altceva

Registru → IF/ID: IC (32) + o comandă (32) = 64
→ ID/EX: sunt date din reg gen (pe 32)
32 (op1) + 32 (op2) + 32 (16→32) + 32 (PC+4) = 128

→ EX/MEM: rez din ALU + adresă ramificație
+ comp operanzi pt ramificație condiționată
(sub op1 op2) = 32·3 + 1

→ MEM/WB: adresa reg + write = 64

+5 pt ele pt a păstra ce mi-a intrat

lw | IF | ID | EX | MEM | WB |

citim din reg → pe dreapta
scriem pe reg → pe stânga

Program Counter → PC+4
→ alta adresă (doar cu instrucțiuni)

ID → citesc din reg IF/ID → PC+4
→ instrucțiune
→ scot operanzi și citesc din reg gen
→ pasez la ID/EX ce aveam în PC
→ extind 16 → 32
→ scriu datele în ID/EX

EX → citesc din reg ID/EX
→ verific tipul de op și văd ce calculez cu ALU
→ se scrie tot ce e calculat în EX/MEM (bitul 0, orice rez pe care l-am calculat)

MEM →

WB → citim din MEM/WB
→ scriem în reg generale oferite de instrucțiune (la tip I aveam 5 biți (rt) pt locație)

Hazard de date

1) $R_7 = R_{12} - R_{15}$ add R_7, R_{12}, R_{15}

2) $R_8 = R_7 - R_{12}$ sub R_8, R_7, R_{12}

EX/MEM ← că e de tip R

1) IF | ID | EX | MEM | WB ⇒ forward-ing

2) IF | ID | EX | MEM | WB

ID/EX ←

1) lw $s0, 20($t1)

2) sub $t2, $s0, $t3

1) IF | ID | EX | MEM | WB

IF | ID | EX | MEM | WB

→ aici am info despre $s0

$s0

$s0

⇒ Nu merge forwarding

Așa că mai stam puțin cu nop/stall

IF | ID | EX | MEM | WB

SOC 20/05/24

Hazarde date → se știe când iese din ALU → Reg EX/MEM

1. sub $2, $1, $3 (RD, RS, RT)
2. and $12, $2, $5 (RD, RS, RT)
3. or $13, $6, $2 (RD, RS, RT)
4. add $14, $2, $12
5. sw $15, 100($2)

→ Hazarde de date pt că WB nu se poate întoarce sincronizat cu asta

Pot să fac Bubble, să aștaaa

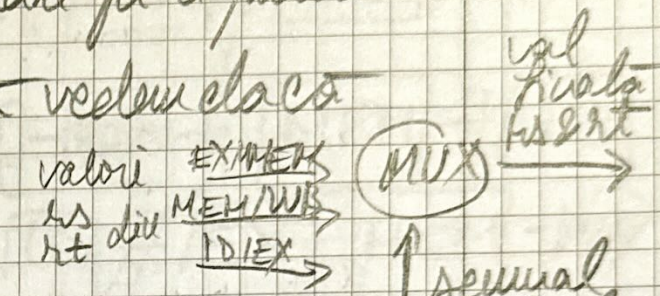
Semnal de control RegWrite activ

RegRD să nu fie 0

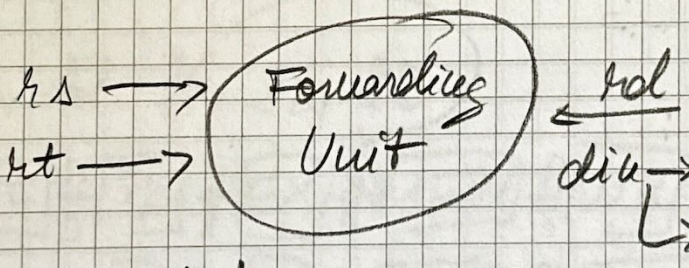
La SW e MemWrite activ și de aia trebuie neapărat să așteptăm

Controlul de Hazard se pun în execuție.

Bagăm MUX-uri pe intrări pt a putea decta Hazard-uri și să vedem dacă putem face forward-ing.



MUX-urile au semnale de control care vin dinspre un forwarding Unit



Forward A/B = 00 → vine din ID/EX ("normal")

Hazard EX/MEM

if - RegWrite RD "1"
- EX/Mem RD (!=) 0
- EX/MEM RD == ID/EX RS/RT

→ Forward A/B 10

Hazard MEM/WB

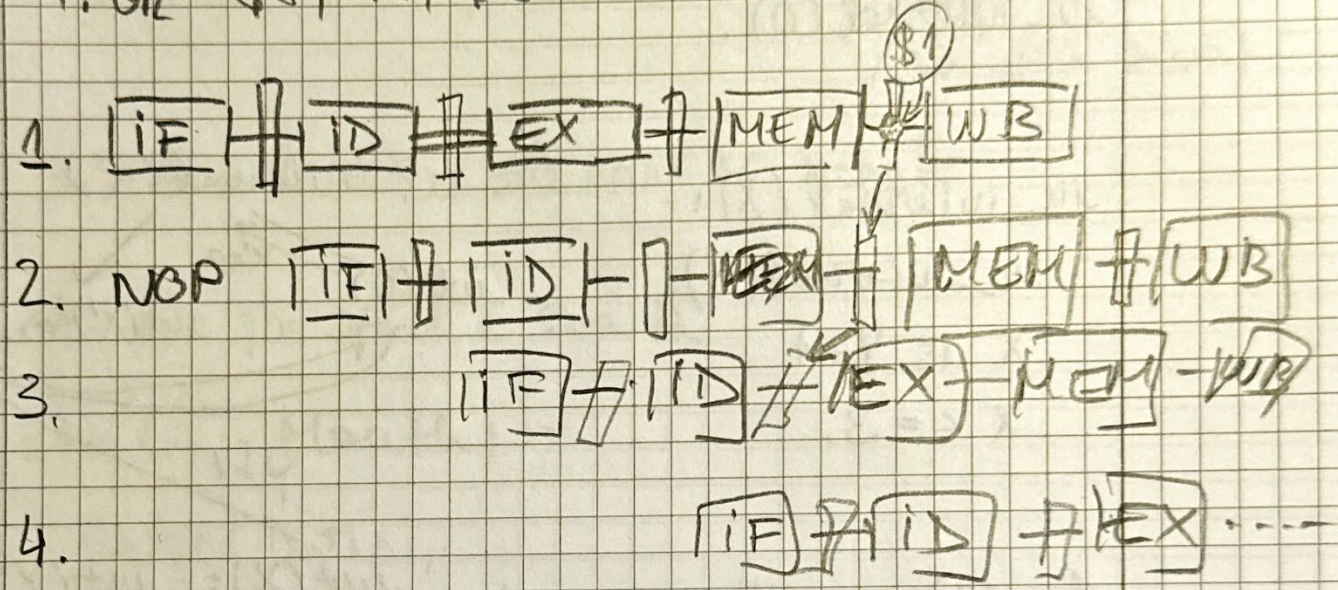
if - RegWrite RD → Forward A/B 01
- ~~EX/Mem~~ MEM/WB RD != 0
- MEM/WB RD == ID/EX RS/RT

~~Seminar~~ 20/05/24

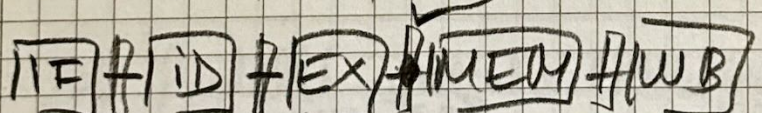
EXERCIȚII → de la bitul 0

1. LW $1, 0($2)
2. SUB $4, $1, $5 ← Hazard
3. AND $6, $1, $7 ← Nu mai e hazard
4. OR $8, $1, $9

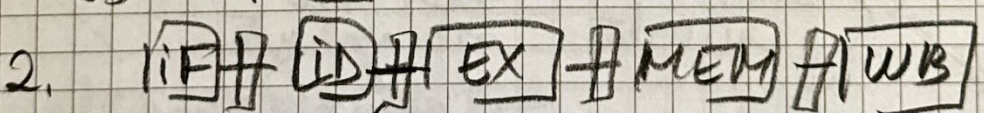
Rezolvăm prin NOP & Forwarding



1. ~~dol~~ $6, $2, $2



2.



↳ Știu adresa din reg general al lui $6 și datele din $1 care s-au mutat